

How This Project Works (and How It Gets Online)

This document explains the Axiom project in simple terms. No deep technical jargon - just the ideas you need to understand what the app does, how the pieces fit together, and how your website ends up live on the internet using a Google Cloud VM, a free domain from dpdns, and nginx.

1. What Is This Project?

Axiom is a web app that helps people research pharmaceutical and medical competition. A user types a question like "What trials are running for diabetes drugs?" and the app goes out to real public websites (ClinicalTrials.gov, FDA data, PubMed papers), pulls live information, shows charts and tables, and uses AI to explain what it found.

Analogy: Think of it like a smart research assistant. You ask a question at the front desk. The assistant runs to the library (public APIs), collects fresh books and articles, organizes them on your desk (charts and tables), and then summarizes what matters.

2. The Big Picture - Four Main Parts

The whole system is made of four cooperating parts:

A. The Frontend (what users see)

Built with Next.js and React. This is every button, page, chart, and form in the browser. It lives in the frontend/ folder. When you open the website, you are talking to this part.

Analogy: The dining room of a restaurant - menus, tables, and what guests actually see and touch.

B. The Backend (the brain)

Built with Python and FastAPI. This is the server that does the real work: talks to the database, calls external APIs, runs the AI agent, and sends results back to the frontend. It lives in the backend/ folder and runs on port 8000 inside Docker.

Analogy: The kitchen - guests never go in, but this is where orders are cooked and decisions are made.

C. The Database (the memory)

PostgreSQL stores things the app needs to remember: user accounts, past investigations, saved summaries, and agent conversation traces. It does NOT store a giant copy of all trial data - that is fetched fresh each time from public APIs.

Analogy: A filing cabinet in the back office. It remembers your customers and their past orders, not every book in the world.

D. External Services (outside help)

The app also talks to services on the internet: ClinicalTrials.gov, openFDA, PubMed for data, OpenRouter for

AI (DeepSeek V4 Flash), and optionally Google for sign-in. These are not part of your code - they are like utilities you plug into.

Analogy: Suppliers and specialists the kitchen calls when it needs something it does not keep on the shelf.

3. How the Pieces Talk to Each Other

Here is the normal flow when someone uses the app:

- User opens the website in their browser (frontend).
- User types a research question and clicks go.
- Frontend sends the request to the backend API.
- Backend asks the AI agent what tools to use.
- Agent fetches live data from ClinicalTrials.gov, FDA, PubMed.
- Backend saves the investigation in PostgreSQL.
- Backend sends charts, tables, and text back to the frontend.
- User sees the dashboard and can ask follow-up questions.

For long AI responses, the backend streams results in real time (like watching text appear word by word) using a technology called SSE.

4. What Is Docker? (And docker compose)

Docker packages each part of the app into a container - a small, self-contained box with everything that part needs to run. docker-compose.yml is the recipe that starts all boxes together.

Analogy: Imagine four lunchboxes, each with a complete meal inside. You do not install ingredients on the kitchen counter - you just open the right lunchbox. Docker does that for software.

Your docker-compose.yml starts three services:

- db - PostgreSQL database
- api - Python/FastAPI backend (port 8000)
- web - Next.js frontend (port 3001 on your machine, 3000 inside)

On your laptop you run: `docker compose up -d --build`. That builds and starts everything. The `-d` flag means run in the background.

5. The .env File (Secret Settings)

The .env file holds passwords and API keys. Docker reads it when starting containers. Important values include:

- OPENROUTER_API_KEY - lets the AI work
- POSTGRES_PASSWORD - database password
- JWT_SECRET - keeps user logins secure
- NEXT_PUBLIC_API_URL - tells the frontend where the backend lives
- GOOGLE_CLIENT_ID / SECRET - for Google sign-in (optional)

Analogy: Like the key ring for your building - each key opens something specific, and you never tape them to the front

door.

6. How Your App Gets Online (Your Setup)

Running on your laptop is not the same as running on the internet. Your setup uses four pieces working together:

A. Google Cloud VM (your rented computer)

A VM (Virtual Machine) is a computer you rent in Google's data center. It has an IP address (like a street address on the internet). You SSH into it, install Docker, copy your project, and run docker compose there - just like on your laptop, but it stays on 24/7.

Analogy: Renting a small office space in a big building. You put your equipment there and leave the lights on so customers can visit anytime.

B. dpdns free domain (your friendly name)

An IP address like 34.123.45.67 is hard to remember. dpdns gives you a free domain name (something like yourapp.dpdns.org) that points to your VM's IP address. You configure this in the dpdns dashboard by setting an A record to your VM's external IP.

Analogy: Like putting a readable sign on your shop instead of telling people "go to building 34, room 123."

C. nginx (the front door receptionist)

nginx is a web server that sits in front of your app. When someone visits your domain, nginx is the first thing that answers. It decides where to send the request:

- Requests for the website (/) go to the Next.js app on port 3001
- Requests for the API (/api or port 8000) go to the FastAPI backend
- Can also handle HTTPS (the padlock in the browser) with SSL certificates

Analogy: A hotel receptionist. Guests walk in the main entrance (port 80/443). The receptionist checks what they need and directs them to the right room - restaurant, gym, or front desk.

D. How a visitor's request flows

When someone types your dpdns domain in Chrome:

- Browser looks up the domain and finds your GCP VM IP (via dpdns DNS).
- Browser connects to nginx on port 80 or 443.
- nginx forwards the request to the correct Docker container port.
- The container (web or api) processes it and sends a response back.
- nginx passes the response to the visitor's browser.
- The user sees your app.

7. Deploying Updates (The Simple Version)

When you change code and want the live site updated:

- Push or copy your latest code to the GCP VM (git pull or scp).

- Make sure .env on the server has the right production values.
- Run: docker compose up -d --build --force-recreate
- Docker rebuilds images and restarts containers with new code.
- nginx keeps running - it just talks to the new containers.

Important: NEXT_PUBLIC_API_URL in .env should be your public API address (e.g. <https://yourdomain.dpdns.org> or <https://yourdomain.dpdns.org/api>) so the browser can reach the backend, not localhost.

8. Project Folder Map (What Lives Where)

- frontend/ - website UI (pages, components, styles)
- backend/ - API server, AI agent, database models
- docker-compose.yml - starts db, api, and web together
- .env - secrets and configuration (never commit to git)
- .env.example - template showing what settings exist
- data/ - optional local data storage

9. Technologies at a Glance

You do not need to master all of these on day one. Just know the names:

- Next.js / React - builds the website
- Tailwind CSS - styling (colors, spacing, layout)
- FastAPI - Python web framework for the API
- PostgreSQL - database
- Docker - runs everything in containers
- nginx - public-facing web server on your VM
- GCP - Google Cloud, hosts your VM
- dpdns - free DNS so your domain points to the VM

10. Common Problems (Simple Fixes)

Website does not load at all

- Check VM is running in GCP console
- Check firewall allows ports 80 and 443 (and 22 for SSH)
- Check nginx is running: `sudo systemctl status nginx`
- Check Docker containers: `docker compose ps`

Page loads but API calls fail

- Check NEXT_PUBLIC_API_URL points to the public URL, not localhost
- Check nginx is proxying API requests to port 8000
- Check api container is healthy: `curl http://localhost:8000/health`

After code changes, nothing changed

- Did you rebuild? `docker compose up -d --build`
- Hard-refresh the browser (Ctrl+Shift+R or Cmd+Shift+R)

11. One-Paragraph Summary

Axiom is a research web app with a React frontend, a Python backend, and a PostgreSQL database, all packaged in Docker. It fetches live medical data and uses AI to explain it. You host it on a Google Cloud VM, point a free dpdns domain at that VM, and use nginx as the front door that directs visitors to the right part of the app. To update the live site, copy new code to the VM and rebuild with docker compose.